밥상 원가 vs 사먹을 때 — 개발 계획서

기능 명세서

기후 식탁 · 2026-06-26

개발 계획서 — 밥상 "재료 원가 vs 사먹을 때" + 소비자 구매단위

문서 종류: 기능 명세 + 개발 계획서
작성일: 2026-06-26
대상 시스템: 기후 식탁 (Climate Table) — "밥상 차리기" 탭 확장
작성: 기후 식탁 개발

---

1. 문서 개요

1.1 목적

"밥상 차리기" 탭은 현재 **재료 원가**(집에서 사다 해먹는 비용)만 보여준다.
본 개발에서는 두 가지를 추가한다.

1. **"밖에서 사먹을 때(외식) 금액"**을 재료 원가와 나란히 표시한다.
2. 재료 가격을 **소비자가 실제로 사는 단위(포기·개·망·단·포대)**로 보여준다.

목표는 "이 날씨에 내 장바구니와 외식비가 얼마가 되는가"를 더 직관적으로 체감하게 만드는 것이다.

1.2 범위

- 대상: `backend/dishes.py`, `backend/app.py`, `backend/test_api.py`, `frontend/app.js`, `frontend/style.css`, `frontend/index.html`(필요 시).
- 비대상: 기존 시뮬레이션 탭·학습 퀴즈 탭(변경 없음), 데이터셋·회귀모델 로직.

1.3 용어

용어	정의
재료 원가(cost)	음식 재료를 사서 직접 해먹을 때의 비용. 작물=도매가×분량, 부재료=정가.
외식가(eatout)	식당에서 사먹을 때의 금액. 음식별 메뉴 기준가 + 기후로 오른 재료비 전가.
구매단위	소비자가 실제 사는 단위(배추=포기, 무·사과=개, 양파=망, 시금치=단, 쌀=포대).
평년 기준(baseline)	극단기상이 없는 BASELINE 기상에서의 가격. 날씨 효과 분리용.
전가	재료비 상승분이 외식 메뉴가에 더해지는 것.

---

2. 배경 및 문제 정의

기존 화면은 밥상 총액을 "재료 원가" 한 가지로만 보여준다. 두 가지 한계가 있다.

- **체감 격차**: 사람들은 재료를 사서 해먹기도 하지만 사먹기도 한다. 외식가를

함께 보여줘야 "기후플레이션이 외식비까지 올린다"는 메시지가 완성된다.

- **단위 괴리**: 원가를 1인분 분량(예: 배추 0.12kg)으로만 계산하면 금액이

작아(수백 원) 체감이 약하다. 소비자는 "배추 한 포기 5,400원"으로 가격을
기억한다. 구매단위 가격을 함께 보여주면 훨씬 와닿는다.

---

3. 기능 명세

3.1 기능 목록

ID	기능	설명	우선순위
F1	외식가 계산	음식별 메뉴 기준가 + 기후 전가로 외식 총액 산출	필수
F2	총액 2종 표시	"재료 원가"와 "사먹을 때"를 결과 패널 상단에 나란히	필수
F3	구매단위 가격	원자재 분해에서 작물별 단위가(포기/개/망/단/포대) 표시	필수
F4	음식별 2종 가격	각 음식 행에 원가/외식가 함께 표기	권장
F5	데이터 분리	외식 단가·구매단위를 dishes.py 한 곳에서 관리	필수
F6	API 확장	/api/dishes·/api/meal 응답에 외식가·단위 정보 추가	필수

3.2 기능 상세

F1. 외식가 계산

- 각 음식에 평년 기준 **메뉴가(menu_base)**를 정의한다.
- 외식가 = `menu_base + (재료원가_예측 − 재료원가_평년)`.
- 평년 날씨에서는 외식가 = menu_base. 폭염 등에서는 재료 상승분만큼 가산.
- 식당이 재료비 오른 만큼 천천히 메뉴가를 올리는 현실을 단순화한 모델.

F2. 총액 2종 표시

- 결과 패널 최상단에 두 줄(또는 두 칼럼):
- 재료 원가(집밥): 강조 큰 숫자 + 평년比 %.
- 사먹을 때(외식): 보조 숫자 + 평년比 %.

F3. 구매단위 가격

- 원자재(기후 작물) 분해의 각 막대에 단위가를 함께:
- 예: `배추 1포기 9,625원 (평년 5,400) · +78%`.
- 단위가 = 도매가(원/kg) × 단위중량(kg). 별도 소매 마진은 적용하지 않는다

(기존 packPrice 관례 유지, 과대표기 방지).

F4. 음식별 2종 가격

- 음식 목록 각 행: `김치찌개 원가 4,400 · 외식 8,249 (+6%)`.

F5. 데이터 분리 / F6. API 확장 → 5절·6절 참조.

---

4. 사용자 시나리오

1. 사용자가 시뮬레이션 탭 또는 밥상 탭 프리셋에서 "2024 폭염·가뭄"을 고른다.
2. 밥상 탭에서 음식(공기밥·김치찌개·배추김치 등)을 식탁에 드래그한다.
3. 결과 패널이 즉시 갱신된다.

- 재료 원가 13,573원 (+7%) / 사먹을 때 41,250원 (+3%)
- 음식별: 김치찌개 원가 4,400 · 외식 8,249 …
- 원자재: 배추 1포기 9,625원 (평년 5,400) · +78% …

1. 사용자는 "직접 해먹어도 오르고, 사먹어도 오른다"를 체감한다.

---

5. 화면 설계 (frontend)

5.1 결과 패널 구성 (위→아래)

```
┌ 총액 영역 ──────────────────────────────┐
│  재료 원가(집밥)   13,573원      +7%    │  ← 큰 강조
│  사먹을 때(외식)   41,250원      +3%    │  ← 보조
└─────────────────────────────────────────┘

  인사이트: "이 날씨에서 배추가 평년比 +78%로 …"

  [음식별]
   공기밥        원가 281 · 외식 1,000      +0%
   김치찌개      원가 4,400 · 외식 8,249     +6%
   …
  [원자재(기후 작물) 가격 영향]
   배추  1포기 9,625원 (평년 5,400)  ███████  +78%
   무    1개 2,400원 (평년 1,220)   ███      +97%
   …
```

5.2 컴포넌트/표시 항목

영역	항목	데이터 출처
총액	재료 원가, 외식가 각각 금액·평년比%	`total.cost`, `total.eatout`
음식별	이름, 원가, 외식가, %	`dishes[]`
원자재	작물명, 구매단위·단위가(평년/예측), %, 사용 음식	`ingredientTotals[]`

5.3 스타일

- 총액 2종: 기존 `.meal-total` 확장. 원가=기존 강조, 외식=보조(작은 글씨·회색).
- 단위가: 기존 `.ing-bar .ib-top` 라인에 단위·평년가 병기.
- 신규 CSS 최소화, 기존 토큰(`var(--*)`) 재사용.

---

6. 데이터 모델 (backend/dishes.py)

6.1 구매단위 (신규 상수 UNITS)

작물	단위	단위중량(kg)	평년 단위가(8월,예시)
배추	포기	2.5	5,400원
무	개	1.0	1,220원
사과	개	0.3	999원
쌀	20kg 포대	20	54,000원
시금치	단	0.4	1,316원
양파	망	1.5	1,665원

```
UNITS = {
    "배추":  {"unit": "포기",      "kg": 2.5},
    "무":    {"unit": "개",        "kg": 1.0},
    "사과":  {"unit": "개",        "kg": 0.3},
    "쌀":    {"unit": "20kg 포대", "kg": 20},
    "시금치": {"unit": "단",        "kg": 0.4},
    "양파":  {"unit": "망",        "kg": 1.5},
}
```

6.2 음식별 외식 단가 (DISHES 각 항목에 eatout 추가)

음식	외식 단가(평년)
공기밥	1,000
김치찌개	8,000
된장찌개	8,000
제육볶음	9,000
배추김치(반찬 단품)	3,000
무생채(반찬 단품)	3,000
시금치나물(반찬 단품)	3,000
양파장아찌(반찬 단품)	2,000

> 외식 단가·구매단위·환산중량은 모두 교육용 근사값이며 `dishes.py` 한 곳에서 조정한다. 레시피(재료·분량)는 기존과 동일.

---

7. 가격 계산 로직 (backend/app.py compute_meal)

7.1 정의

- 작물 재료비(원가) = `predict(작물, 시나리오, 월) × 분량(kg)`.
- 부재료(고정) = `won` 그대로(예측=평년 동일).
- 음식 원가 cost = Σ(작물 재료비) + Σ(고정 재료비).
- 음식 외식가 eatout_pred = `menu_base + (cost_pred − cost_base)`,

eatout_base = `menu_base`.

- 단위가 unitPrice = `predict(작물, 시나리오, 월) × UNITS[작물].kg`.
- 변동률 pct = `round((예측 − 평년) / 평년 × 100)`, 평년=0이면 0.

7.2 계산 예시 — 김치찌개 (8월, 2024 폭염 프리셋)

재료: 배추 0.12kg · 양파 0.05kg · 무 0.03kg · 돼지고기 2,500 · 두부 900 · 양념 400

항목	평년	폭염
배추 (도매×0.12)	259원	462원
양파 (×0.05)	56원	66원
무 (×0.03)	37원	72원
고정 재료 합	3,800원	3,800원
재료 원가(cost)	**4,151원**	**4,400원**
외식가(eatout)	**8,000원**	**8,249원**(=8,000+249)

→ 재료 원가는 +6%, 외식가는 +3% 상승. 폭염이 집밥·외식 모두를 올리지만,
외식가는 메뉴 기준가가 커서 상승률이 완만하다(현실적).

7.3 작물 단위가 예시 (8월)

작물	평년 단위가	폭염 단위가	변동
배추 1포기	5,400원	9,625원	+78%
무 1개	1,220원	2,400원	+97%
양파 1망	1,665원	1,980원	+19%
시금치 1단	1,316원	1,532원	+16%
사과 1개	999원	1,170원	+17%
쌀 20kg 포대	54,000원	56,200원	+4%

---

8. API 명세

8.1 GET /api/dishes

- 각 음식에 `eatout`(외식 기준 단가) 추가.
- 응답에 `units`(구매단위 표) 포함.

```
{
  "dishes": [
    {"id":"kimchi_jjigae","name":"김치찌개","img":"kimchi_jjigae.png","emoji":"🥘",
     "eatout":8000,
     "ingredients":[{"name":"배추","kind":"crop","crop":"배추","amount_kg":0.12}, ...]}
  ],
  "units": {"배추":{"unit":"포기","kg":2.5}, ...}
}
```


8.2 POST /api/meal

요청(기존 동일):


```
{ "scenario": {"heat_days":28,"drought_idx":75,...}, "month":8,
  "dishes": ["rice","kimchi_jjigae","baechu_kimchi"] }
```


응답(확장):


```
{
  "total": {
    "cost":   {"predicted":13573,"baseline":12680,"pct":7},
    "eatout": {"predicted":41250,"baseline":40000,"pct":3}
  },
  "dishes": [
    {"id":"kimchi_jjigae","name":"김치찌개",
     "cost":{"predicted":4400,"baseline":4151,"pct":6},
     "eatout":{"predicted":8249,"baseline":8000,"pct":3},
     "ingredients":[ ...기존 동일... ]}
  ],
  "ingredientTotals": [
    {"crop":"배추","unit":"포기","unitKg":2.5,
     "unitPrice":{"predicted":9625,"baseline":5400,"pct":78},
     "predicted":1155,"baseline":648,"pct":78,
     "usedIn":["김치찌개","배추김치"]}
  ]
}
```


8.3 응답 필드 설명

필드	타입	설명
total.cost / total.eatout	obj	밥상 총 원가 / 외식가 (predicted·baseline·pct)
dishes[].cost / .eatout	obj	음식별 원가 / 외식가
ingredientTotals[].unit	str	구매단위 라벨(포기/개/망/단/포대)
ingredientTotals[].unitKg	num	단위 환산중량(kg)
ingredientTotals[].unitPrice	obj	구매단위 가격(predicted·baseline·pct)
ingredientTotals[].predicted/baseline	num	밥상 내 해당 작물 사용분 원가

> ⚠ 호환성: 기존 `total.predicted`(평면)에서 `total.cost.predicted`(중첩)로 변경된다. 프론트 `renderMeal` 도 함께 수정해야 한다(7번 작업).

---

9. 개발 계획 (작업 분해)

각 작업은 독립적으로 테스트·검토 가능한 단위다. 백엔드는 `python backend/test_api.py`로 검증, 프론트는 브라우저 수동 QA.

작업 1 — 데이터 모델 추가 (dishes.py)

- 파일: `backend/dishes.py`
- 내용: `UNITS` 상수 추가, `DISHES` 각 항목에 `eatout` 값 추가.
- 산출물: 데이터만. 기존 구조 유지.
- 검증: import 후 8종 모두 `eatout>0`, UNITS 6작물 모두 존재.

작업 2 — 계산 로직 확장 (app.py compute_meal)

- 파일: `backend/app.py`
- 내용: `compute_meal` 반환을 cost/eatout 중첩 구조로 변경, 단위가 산출 추가.

`GET /api/dishes`에 `units`·`eatout` 포함.

- 산출물: 새 응답 스키마(8절).
- 검증(test_api.py 확장):
- `/api/dishes`에 `eatout`·`units` 존재.
- `/api/meal` 폭염: `total.cost.pct>0`, `total.eatout.predicted ≥ baseline`.
- 평년: `cost.pct≈0`, `eatout.pct≈0`.
- `ingredientTotals[].unit/unitPrice` 존재, 빈 밥상=0.

작업 3 — 프론트 렌더 수정 (app.js)

- 파일: `frontend/app.js`
- 내용: `renderMeal`을 새 스키마(`total.cost` / `total.eatout`, `ingredientTotals[].unitPrice`)에 맞게 수정. 총액 2종·음식별 2종·단위가 표시.
- 검증(수동): 폭염 프리셋에서 두 총액·음식별·단위가가 올바로 표시, 빈 밥상 안내.

작업 4 — 스타일 (style.css)

- 파일: `frontend/style.css`
- 내용: `.meal-total` 2행 레이아웃, 외식가 보조 스타일, 단위가 라인 스타일.
- 검증(수동): 데스크톱·모바일(≤833px)에서 레이아웃 정상.

작업 5 — 통합 검증 & 문서

- `python backend/test_api.py` 전체 통과(회귀 없음).
- 브라우저 QA 체크리스트(10절) 통과.
- README 표/아키텍처에 외식가·단위 반영(필요 시).

작업 순서·의존성

작업1 → 작업2 → 작업3 → 작업4 → 작업5 (순차). 작업1·2는 백엔드,
3·4는 프론트로 분리되어 각 단계가 독립 검증된다.

구현 방식

서브에이전트 주도(각 작업마다 구현 에이전트 + 스펙/품질 리뷰), 마지막에
전체 리뷰. 모든 변경은 작은 커밋으로 분리한다.

---

10. 테스트 계획

10.1 백엔드 자동 (test_api.py)

케이스	기대
GET /api/dishes — eatout·units	8종 eatout>0, units 6작물
POST /api/meal — 폭염	total.cost.pct>0, eatout.predicted≥baseline
POST /api/meal — 평년	cost.pct≈0, eatout.pct≈0
빈 밥상	cost/eatout 모두 0
미지 음식 id	스킵, 유효분만 계산
ingredientTotals	unit/unitKg/unitPrice 존재
회귀	기존 29개 케이스(평면→중첩 키 수정 포함) 전부 통과

10.2 프론트 수동 QA 체크리스트

1. 밥상 탭 진입 → 음식 드래그/탭 추가 정상.
2. 결과 패널에 **재료 원가·사먹을 때** 두 총액 표시.
3. 음식별 행에 원가·외식가 병기.
4. 원자재 막대에 구매단위 가격(예: 배추 1포기 9,625원) 표시.
5. 프리셋 전환·날씨 공유 시 두 총액 모두 갱신.
6. 빈 밥상 안내, 모바일 폭 레이아웃 정상.
7. 기존 시뮬레이션·퀴즈 탭 회귀 없음.

---

11. 엣지 케이스 / 예외 처리

- 빈 밥상: cost/eatout 0, pct 0.
- 평년 시나리오: 두 pct ≈ 0(전가분 0).
- 풍작(가격 하락): 외식 전가분 음수 허용 → 외식가가 메뉴가보다 소폭 하락.
- 알 수 없는 음식 id: 무시(기존과 동일).
- 분량 0/정가 0 방어, 0 나눗셈 방어(pct).

---

12. 비범위 (YAGNI)

- 지역별 외식가 차이, 인건비·임대료 별도 모델.
- 수량 스텝퍼 UI, 영수증 출력, 저장/공유.
- 소매 유통마진 별도 계수(도매×단위중량 관례 유지).

---

13. 튜닝 포인트 / 오픈 이슈

1. **외식 단가 값**(6.2 표) — 가게 기준에 맞춰 조정 가능.
2. **외식가 전가 방식** — "재료 상승분 전액 가산" 채택. 대안: 배수(원가×3), 부분 전가(상승분×k).
1. **구매단위·환산중량** — 기존 UNIT_PACK과 동일(쌀은 20kg 포대로 lumpy).
2. **호환성** — 응답 평면 키 → 중첩 키 변경에 따른 프론트 동시 수정 필요.